QCAA HW1

林威橙 (B13202050)

April 16, 2026

The implementation code for this homework is available on GitHub (Ref. [7]):
https://github.com/daniellin54057/QCAA-HW1

1 Problem 1

1.1 Plot the training loss and test loss versus epoch for your best model configuration.

We selected the model in Fig. 2(d), Case 4, as our best-performing configuration, since it achieved the most satisfactory overall performance among all tested settings. The training loss and test loss versus epoch for this model are shown in Fig. 1.

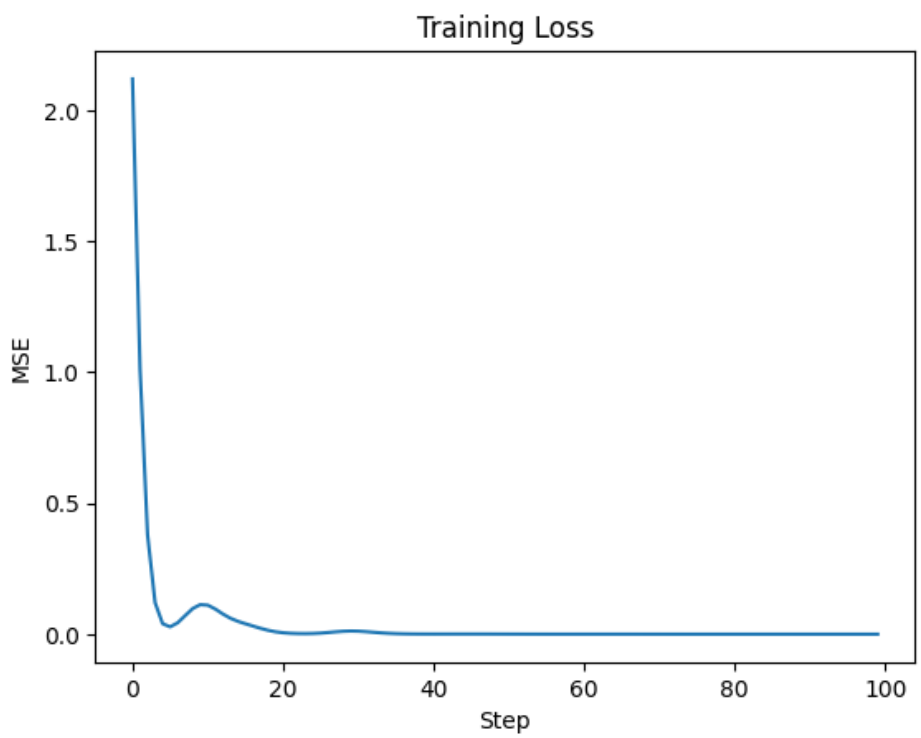


Figure 1: Training loss and test loss versus epoch for the best model configuration, corresponding to Fig. 2(d) Case 4.

From Fig. 1, the loss decreases rapidly during the early stage of training and then gradually stabilizes. This behavior indicates that the model can efficiently learn the dominant structure of the

target function. The later-stage flattening of the curve suggests that the optimization has largely converged.

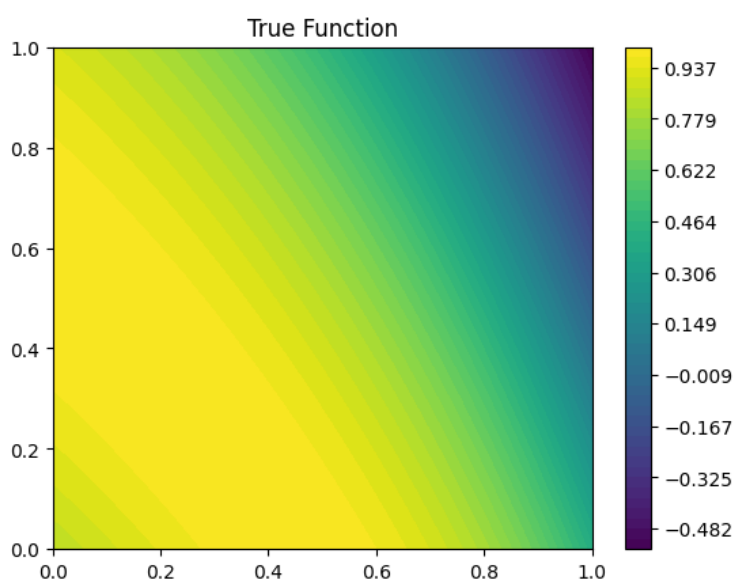
1.2 Provide a comparison table of at least 4 different hyperparameter configurations (e.g., number of layers, number of qubits, encoding gate type), reporting train MSE, test MSE, and the number of trainable parameters for each.

In this part, I investigate six different combinations of circuit depth and number of qubits, using 1, 3, and 5 layers with 1 and 2 qubits. I then compare their training loss, test loss, and prediction plots.

Table 1: Comparison of different circuit configurations

Case	Number of Layers	Number of Qubits	Train Loss	Test Loss
1	1	1	0.00068	0.38602
2	1	2	0.00063	0.12189
3	3	1	0.00209	0.04170
4	3	2	0.00004	0.07507
5	5	1	0.00021	0.06328
6	5	2	0.00021	0.21253

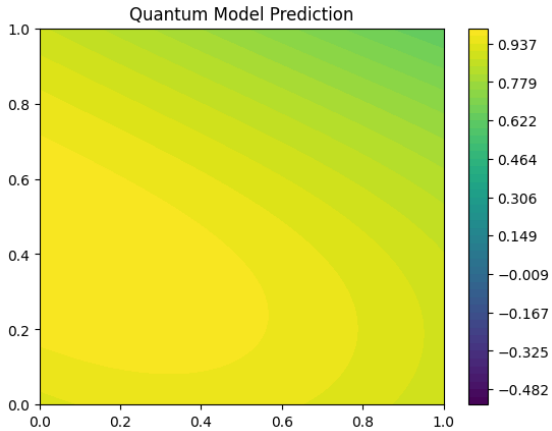
Table 1 shows that increasing circuit complexity does not automatically improve generalization. In particular, shallow circuits with too few qubits cannot model the target function well, while deeper or larger circuits may fit the training data extremely well but still show worse test performance. Among these configurations, intermediate-depth circuits provide a better balance between expressivity and generalization.



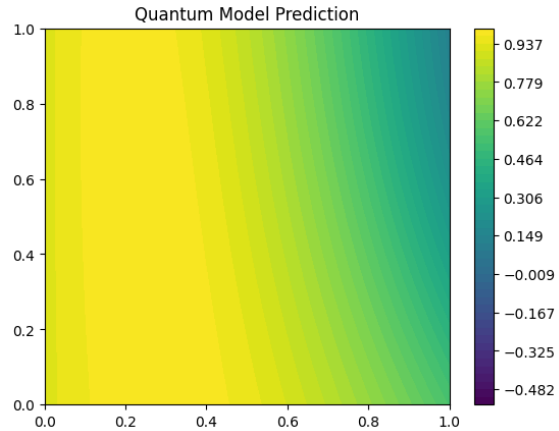
(a) Original Function

Figure 2: Ground truth function

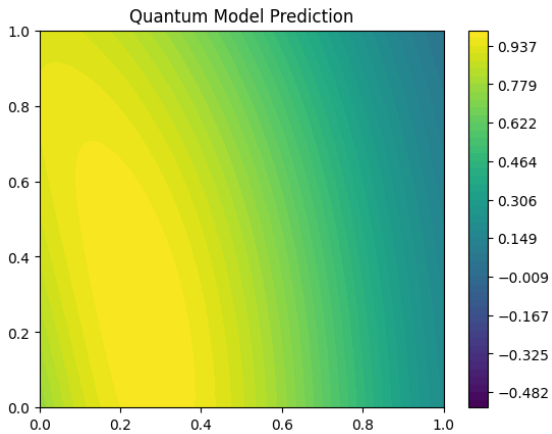
Figure 2 presents the target function over the full domain. The function is smooth overall, but it still contains nontrivial variation across the input space, so a successful model must capture both the global trend and part of the finer local structure.



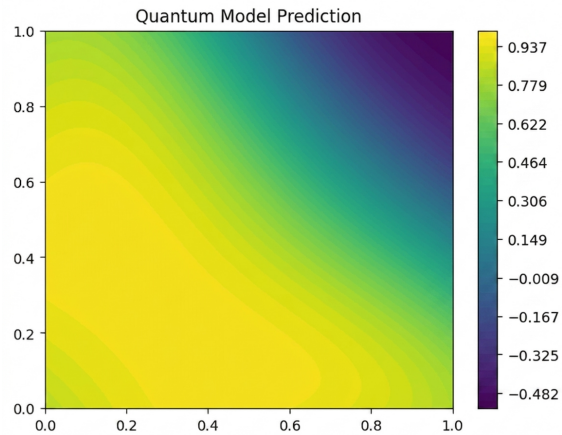
(a) Case 1



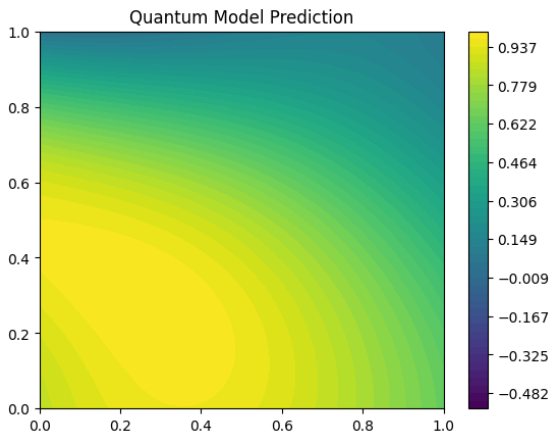
(b) Case 2



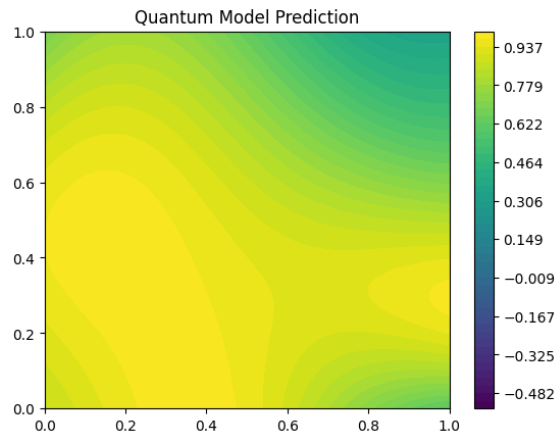
(c) Case 3



(d) Case 4



(e) Case 5



(f) Case 6

Figure 3: Prediction plots for the six circuit configurations

Figure 3 provides a visual comparison of the learned functions. Cases 1 and 2 are too simple to reproduce the target shape accurately, indicating insufficient expressivity. Cases 3–5 capture the

overall trend more successfully, while Case 6 becomes less stable in test performance despite its larger circuit size. This again suggests that a more expressive circuit is not always better if the generalization ability deteriorates.

1.3 Compute and plot the Fourier spectrum of both the target function and your trained model ' s output over the test domain. Discuss which frequency components your model captures and how this relates to the circuit depth, referencing the analysis in Ref. [1].

To compare the spectral behavior of the target function and the trained quantum model, we evaluate both on the test domain and compute their discrete Fourier spectra. Since the zero-frequency component can dominate the visualization, we also examine the mean-removed spectra in order to better reveal the nontrivial frequency structure.

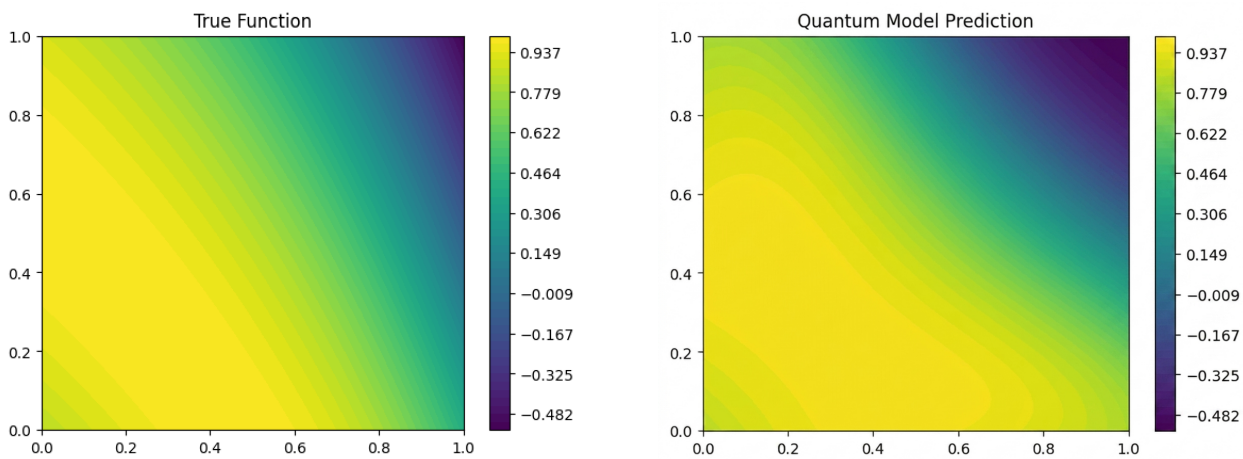


Figure 4: Function values over the test domain. Left: target function. Right: trained model output.

Figure 4 shows the target function and the trained model output over the test domain. The model reproduces the overall shape of the target function reasonably well, indicating that it captures the dominant large-scale variation. In particular, the broad gradient across the domain is preserved, although some local details are visibly smoothed out.

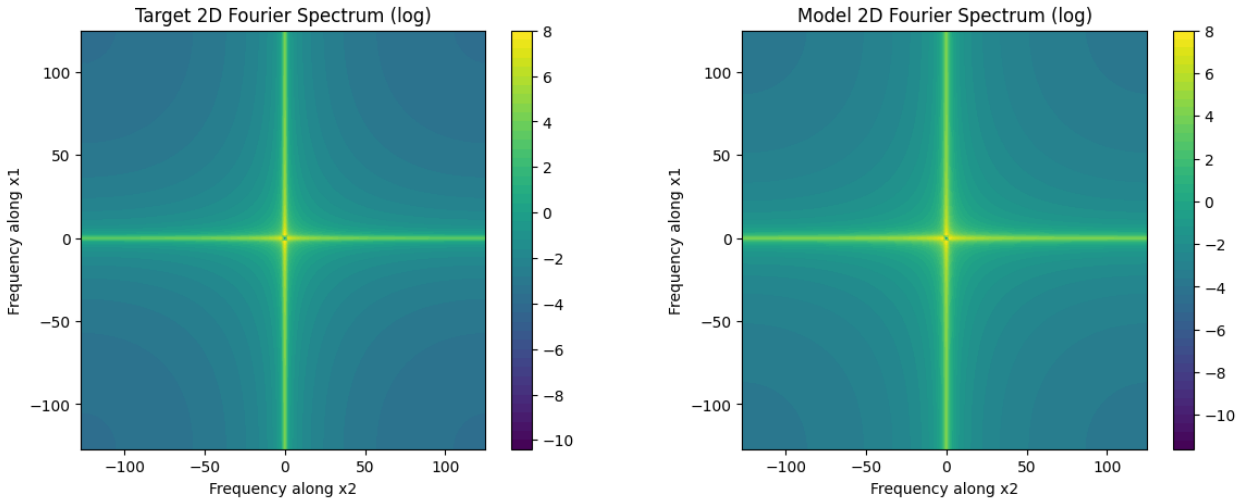


Figure 5: Two-dimensional Fourier spectra on the test domain after mean removal. Left: target spectrum. Right: model spectrum.

Figure 5 compares the two-dimensional Fourier spectra. The dominant spectral weight is concentrated near low frequencies, which indicates that both functions are mainly governed by slowly varying components. The trained model captures the main low-frequency structure of the target spectrum, but the spectral distribution of the model is more concentrated and less rich in weaker components.

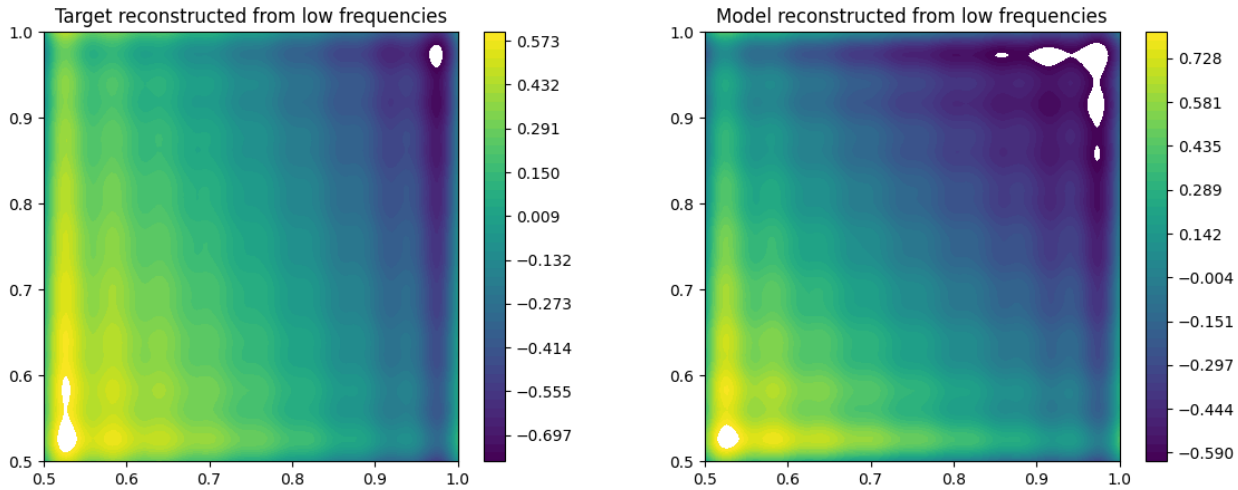


Figure 6: Zoomed-in view of the low-frequency region of the Fourier spectra. Left: target spectrum. Right: model spectrum.

The zoomed-in spectra in Figure 6 make the low-frequency differences easier to observe. The model reproduces the strongest nearby modes, but some weaker surrounding components are suppressed or missing. This suggests that the circuit learns the most important spectral content first, while the finer structure remains more difficult to recover.

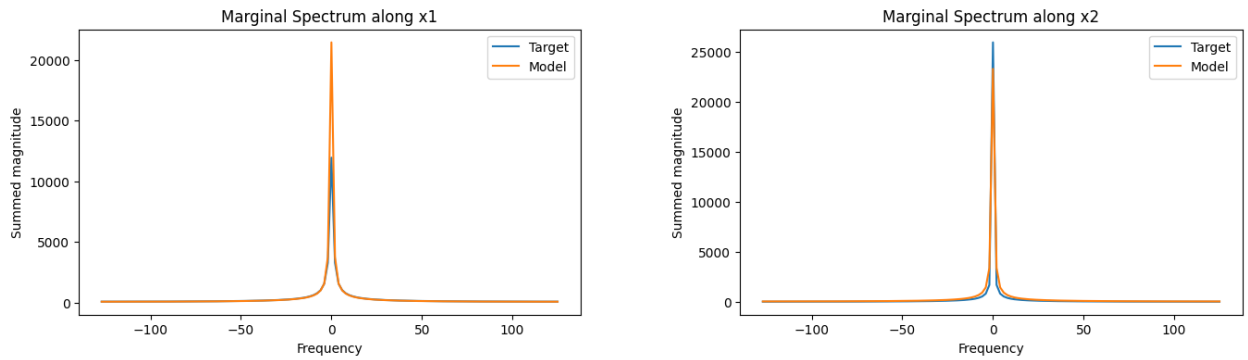


Figure 7: Marginal Fourier spectra along the x_1 and x_2 directions.

Figure 7 shows the one-dimensional marginal spectra along each coordinate direction. These plots confirm that the model captures the main peaks in the low-frequency region, while higher-frequency or weaker components are less accurately represented.

Overall, the trained model captures the most important low-frequency components but does not fully reproduce the weaker or more detailed spectral structure. This observation is consistent with the Fourier analysis of data-reuploading quantum circuits discussed in Ref. [1]: the accessible frequency content is constrained by the encoding structure, while increasing the circuit depth enriches the set of representable frequency components. Therefore, with the present finite circuit depth, the model has sufficient expressive power to learn the dominant low-frequency behavior, but not enough to fully recover all finer spectral details of the target function.

2 Problem 2

2.1 Try to obtain Fig. 6 in Ref. [3] with the performance on the circle dataset.

We reproduced Fig. 6 in Ref. [3] for the circle dataset. The resulting performance comparison is shown in Fig. 8.

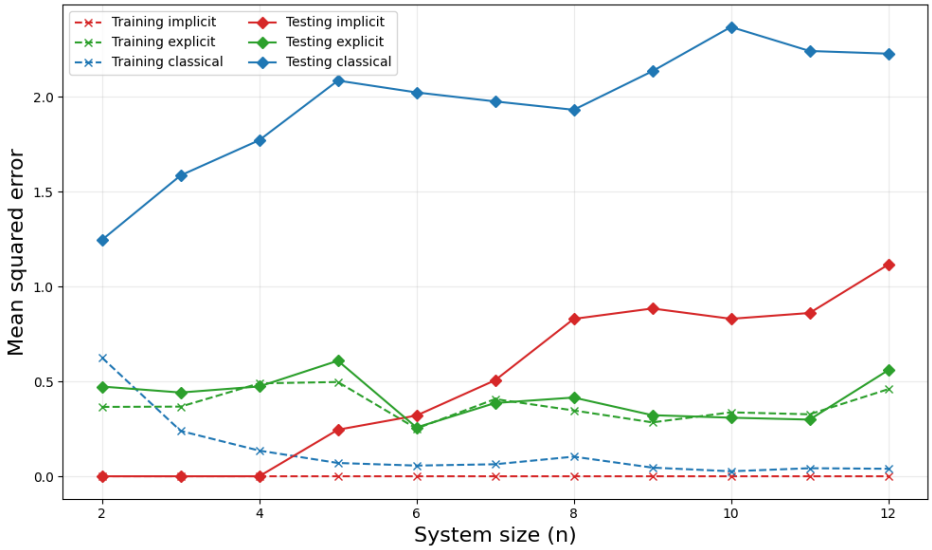


Figure 8: Performance on the circle dataset, reproduced in the style of Fig. 6 in Ref. [3].

Figure 8 compares the training and testing mean squared errors as the system size increases. The reuploading method maintains consistently lower errors than the other approaches, while the classical kernel method shows significantly larger errors over most system sizes. This already suggests that the reuploading architecture has a stronger inductive bias for learning the circle structure.

2.2 Plot the decision boundaries of all three methods on both datasets (6 plots total, arranged as a 3×2 grid).

Figure 9 shows the decision boundaries of the three methods on the two datasets, arranged as a 3×2 grid in one combined figure.

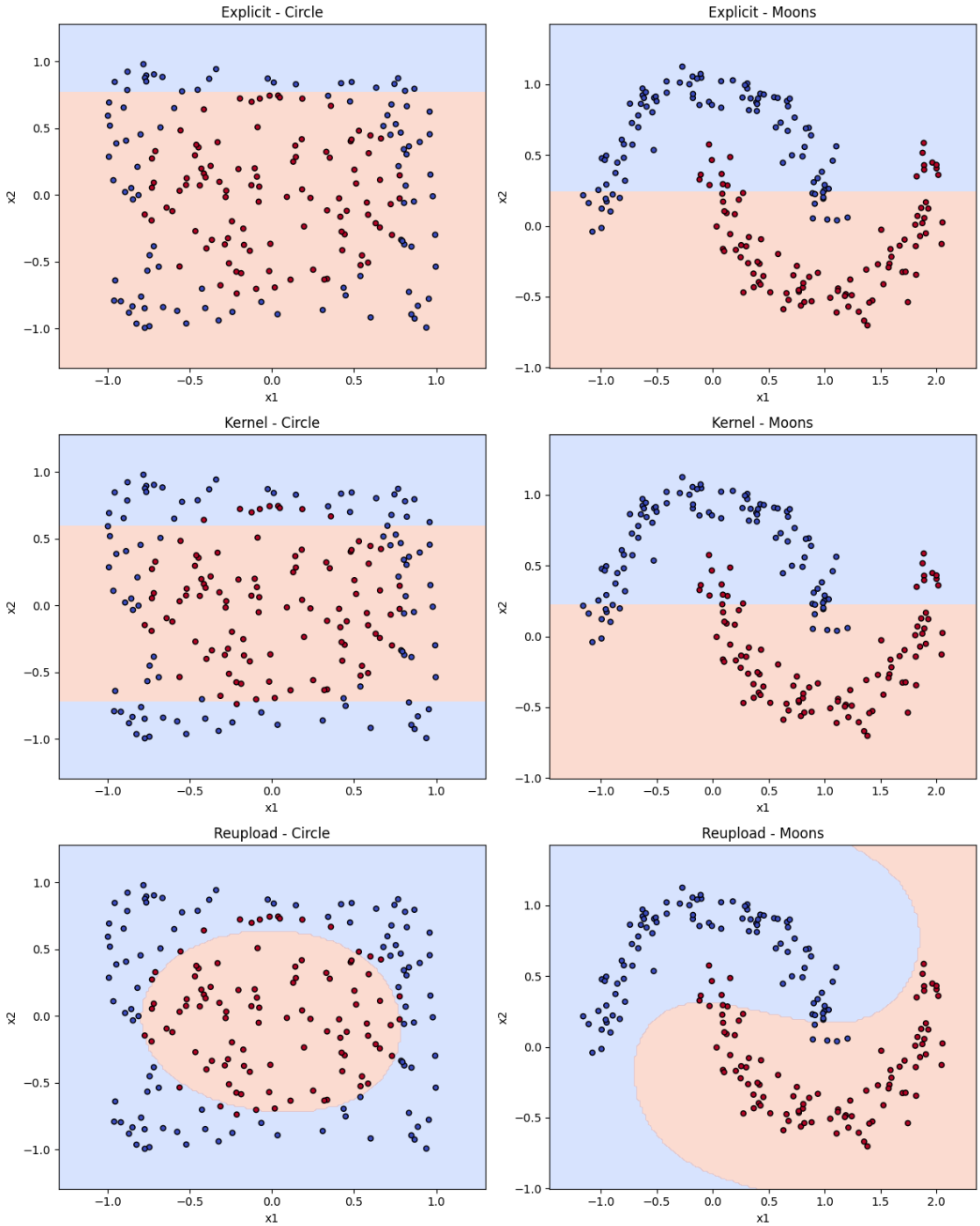


Figure 9: Decision boundaries of the three methods on the circle and moons datasets, arranged as a 3×2 grid.

From Fig. 9, the difference between the three models is visually clear. On the circle dataset, the reuploading method is the only one that forms a boundary with a shape close to the target circular structure. On the moons dataset, the reuploading method also adapts more flexibly to the nonlinear separation, whereas the explicit and kernel approaches produce simpler and less accurate boundaries.

2.3 Provide a comparison table reporting, for each method on each dataset: test accuracy, number of trainable parameters (or kernel evaluations), and training time.

Table 2 summarizes the test accuracy, model complexity, and training time of the three methods on both datasets.

Table 2: Comparison of the three methods on the circle and moons datasets.

Dataset	Method	Test Accuracy (%)	Trainable Parameters / Kernel Evaluations	Training Time (s)
Circle	Reupload	85.00	9	52.7249
Circle	Explicit	53.33	9	46.9165
Circle	Kernel	75.00	19600	11.5013
Moons	Reupload	90.00	9	52.5687
Moons	Explicit	78.33	9	46.8371
Moons	Kernel	78.33	19600	11.3515

Although the kernel method is faster in training time, it requires a large number of kernel evaluations. In contrast, the reuploading model uses only a small number of trainable parameters while still achieving the best test accuracy on both datasets. This indicates that its stronger representational flexibility is obtained in a parameter-efficient way.

2.4 Write a brief discussion (3–5 sentences) comparing the three methods across both datasets. Relate your findings to the arguments in Ref. [3].

Across both datasets, the reuploading model achieves the best performance and also produces the most appropriate decision boundaries. The explicit model performs worst on the circle dataset and is consistently less flexible in representing nonlinear structures. The kernel method performs better than the explicit model in some cases, but its boundaries are still less adaptive than those of the reuploading approach. These results are consistent with the arguments in Ref. [3], which suggest that data reuploading enhances expressivity and allows compact quantum models to represent more complex classification functions.

3 Problem 3

Due to time constraints, I was not able to complete Problem 3 in this submission. Therefore, I do not report final results, training curves, or quantitative comparisons for this part.

References

[1] Maria Schuld, Ryan Sweke, and Johannes J. Meyer, *The effect of data encoding on the expressive power of variational quantum-machine-learning models*, Physical Review A, 103, 032430 (2021).

[2] Maria Schuld and Johannes Jakob Meyer, *Quantum models as Fourier series*, PennyLane demo, 2020. https://pennylane.ai/qml/demos/tutorial_expressivity_fourier_series

[3] *QCAA HW1 Handout / Experiment Handout_0306.pdf*, course handout for Homework 1.

[4] Sofiene Jerbi, Lukas J. Fiderer, Hendrik Poulsen Nautrup, Jonas M. Kübler, Hans J. Briegel, and Vedran Dunjko, *Quantum machine learning beyond kernel methods*, Nature Communications, 14(1), January 2023.

[5] Shahnawaz Ahmed, *Data-reuploading classifier*, PennyLane demo, 2020. https://pennylane.ai/qml/demos/tutorial_data_reuploading_classifier

[6] Adrián Pérez-Salinas, Alba Cervera-Lierta, Elies Gil-Fuster, and José I. Latorre, *Data Re-Uploading for a Universal Quantum Classifier*, Quantum, 4, 226 (2020). https://doi.org/10.22331/q-2020-02-06-226

[7] Lin, Wei-Cheng, *QCAA HW1 source code repository*, GitHub repository. https://github.com/daniellin54057/QCAA-HW1